

Dynamic Programming Problems

Algorithms Assignment

May 2, 2025

Dynamic Programming Problems

- 6.7.** A subsequence is palindromic if it is the same whether read left to right or right to left. For instance, the sequence $A, C, G, T, G, T, C, A, A, A, A, T, C, G$ has many palindromic subsequences, including A, C, G, C, A and A, A, A, A (on the other hand, the subsequence A, C, T is not palindromic). Devise an algorithm that takes a sequence $x[1 \dots n]$ and returns the (length of the) longest palindromic subsequence. Its running time should be $O(n^2)$.
- 6.17.** Given an unlimited supply of coins of denominations $x_1, x_2, \dots, x_n$, we wish to make change for a value v ; that is, we wish to find a set of coins whose total value is v . This might not be possible: for instance, if the denominations are 5 and 10 then we can make change for 15 but not for 12. Give an $O(nv)$ dynamic-programming algorithm for the following problem.
- Input:** $x_1, \dots, x_n; v$.
- Question:** Is it possible to make change for v using coins of denominations $x_1, \dots, x_n$?
- 6.18.** Consider the following variation on the change-making problem (Exercise 6.17): you are given denominations $x_1, x_2, \dots, x_n$, and you want to make change for a value v , but you are allowed to use each denomination at most once. For instance, if the denominations are 1, 5, 10, 20, then you can make change for $16 = 1 + 15$ and for $31 = 1 + 10 + 20$ but not for 40 (because you can't use 20 twice).

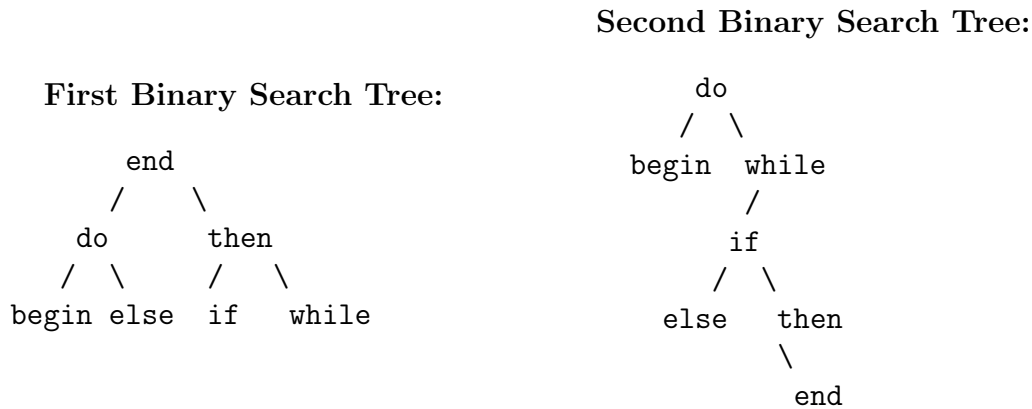


Figure 1: Two binary search trees for the keywords of a programming language.

Input: Positive integers $x_1, x_2, \dots, x_n$; another integer v .

Output: Can you make change for v , using each denomination x_i at most once?

Show how to solve this problem in time $O(nv)$.

- 6.19.** Here is yet another variation on the change-making problem (Exercise 6.17). Given an unlimited supply of coins of denominations $x_1, x_2, \dots, x_n$, we wish to make change for a value v using at most k coins; that is, we wish to find a set of $\leq k$ coins whose total value is v . This might not be possible: for instance, if the denominations are 5 and 10 and $k = 6$, then we can make change for 55 but not for 65. Give an efficient dynamic-programming algorithm for the following problem.

Input: $x_1, \dots, x_n; k; v$.

Question: Is it possible to make change for v using at most k coins, of denominations $x_1, \dots, x_n$?

- 6.20. Optimal binary search trees.** Suppose we know the frequency with which keywords occur in programs of a certain language, for instance:

begin	5%
do	40%
else	8%
end	4%
if	10%
then	10%
while	23%

We want to organize them in a binary search tree, so that the keyword in the root is alphabetically bigger than all the keywords in the left subtree and smaller than all the keywords in the right subtree (and this holds for all nodes).

Figure 6.12 has a nicely-balanced example on the left. In this case, when a keyword is being looked up, the number of comparisons needed is at most three: for instance, in finding **while**, only the three nodes **end**, **then**, and **while** get examined. But since we know the frequency with which keywords are accessed, we can use an even more fine-tuned cost function, the average number of comparisons to look up a word. For the search tree on the left, it is

$$\begin{aligned}\text{cost} &= 1(0.04) + 2(0.40 + 0.10) + 3(0.05 + 0.08 + 0.10 + 0.23) \\ &= 2.42\end{aligned}$$

By this measure, the best search tree is the one on the right, which has a cost of 2.18.

Give an efficient algorithm for the following task.

Input: n words (in sorted order); frequencies of these words: $p_1, p_2, \dots, p_n$.

Output: The binary search tree of lowest cost (defined above as the expected number of comparisons in looking up a word).